

You can see in Table 1 that new functions are of two types - static and prototype. Static functions are available without an instance variable, whereas a prototype function should have a string instance. This will be clear in the following examples.

The *String.raw* function

This function is introduced along with template strings (refer to Part 1 of this series). Template strings allow variable substitution, and the interpretation of special characters like newline, tab, etc. During string operations, it is required to search or compare characters in the exact raw form instead of interpreted forms. The *String.raw* function keeps the string the same as what is 'seen' by the programmer in the editor. In other words, '\n' is two characters '\ ' and 'n'.

String.raw is a static function, which means you do not need to have a string object created to use the function.

The raw function has two forms:

```
String.raw`template string`
String.raw({ raw: 'string'}, substitute_string)
```



Note: The first form does not have parenthesis like other functions. It is just a template string enclosed in back-ticks. In the second form the property name should be 'raw'.

```
1. let year = "2015";
2. console.log(`osidays\t${year}`); //osidays 2015
3. console.log(String.raw`osidays \t${year}`); //osidays \
  t2015
4. console.log(String.raw({ raw: 'osi', '1', '2'})); //
  o1s2i
5. console.log('osi \u083D\uDE80'); //osi
6. console.log(String.raw`osi \u083D\uDE80`); //osidays \
  u083D\uDE80
```

To see the special characters in the above code snippet, please visit, <http://www.es6fiddle.net/1fxcl0ay/>.

Line 2 is an example of a template string. Line 3 displays the non-printable tab character in the raw form. Line 4 is the second form of a raw function where it takes two arguments. The first is the string parameter and the second is a substitution string. Line 5 prints special Unicode characters. In line 6, the same string with *String*.raw displays individual characters.

Finding string and *repeat*

New helper string functions introduced in ES6 are *startsWith()*, *endsWith()* and *includes()*. These three functions need objects to be created, before they can be used. They also take optional second parameters to indicate from which character position the *find* operation should start.

```
let str = "osidays2015";
console.log(str.startsWith("osi")); // true
console.log(str.startsWith("si", 2)); // false
console.log(str.endsWith("15")); // true
console.log(str.endsWith("20")); // false
console.log(str.includes("day")); // true
```

The next utility function is *repeat()*, which takes an argument and returns the same string 'count' a number of times.

```
let pattern = "====";
console.log(pattern.repeat(5)); //=====
console.log('ab'.repeat(5)); //ababababab
```

Number object extensions

The already existing global functions, *isFinite()*, *isNaN()*, *parseFloat()* and *parseInt()*, are now added to the 'Number' object. This groups related functions under the 'Number' object. One difference we should be aware of is that the *global* function automatically converts a number in the string form to a number, whereas when used with the *Number* object, it does not. The following example makes this clear:

```
console.log(Number.isFinite(3)); // true
console.log(Number.isFinite('3')); // false
console.log(isFinite('3')); // true
```

In addition to functions, there are three properties added to the *Number* object. They are: *EPSILON*, *MIN_SAFE_INTEGER* and *MAX_SAFE_INTEGER*.

JavaScript stores numbers as floating point numbers internally. Arithmetic operations may not yield the desired results due to a difference in small precision. For example, $0.21 + 0.2 \neq 0.41$ due to precision error.

The *Number.EPSILON* is the smallest interval between two representable numbers. Its value is $2.220446049250313e-16$.

To overcome precision errors, *Number.EPSILON* is used. When a math operation is performed, check if the resultant difference is more than *Number.EPSILON*. As we are aware, JavaScript has number representation that is in the range of 2^{-53} and 2^{53} .

```
console.log(Number.MIN_SAFE_INTEGER); // -9007199254740991
console.log(Number.MAX_SAFE_INTEGER); // 9007199254740991
```

The new properties help in checking the valid range after mathematical computations. There is a function *isSafeInteger()* that checks if a number is within valid range.

```
console.log(Number.isSafeInteger(42342343434343)); // true
```